© Victor Tan 2025

Angular Workshop
Lab 3
Built-in Directives

1 LAB SETUP 1

2 GENERATING A NEW ANGULAR PROJECT 1

3 NGCLASS 2

3.1 DIFFERENCE BETWEEN [CLASS] AND [NGCLASS] 3

4 NGSTYLE 3

5 USING @IF AND @ELSE (ANGULAR 17 AND LATER) 4

6 USING @FOR (ANGULAR 17 AND LATER) 4

6.1 ACCESSING INDEX OF ITEM IN COLLECTION 5

6.2 STYLING USING FIRST, LAST, ODD AND EVEN LOOP CONTEXTUAL VARIABLES 5

6.3 @FOR WITH CHILD COMPONENTS 6

6.4 PASSING VALUES FROM CHILD COMPONENTS BACK TO PARENT COMPONENT 6

7 COMBINING @IF AND @FOR 7

7.1 SWITCHING SEQUENCE OF @IF AND @FOR 7

8 USING @SWITCH, @CASE AND @DEFAULT (ANGULAR 17 AND LATER) 7

9 MIGRATING FROM OLDER DIRECTIVES TO NEW @IF, @FOR AND @SWITCH SYNTAX 8

1 Lab setup

The setup here is identical to that in Lab 1

2 Generating a new Angular project

The source code for this lab can be found in the `changes` subfolder of `Directives-Demo`

In either a separate command prompt (or shell terminal) or in an embedded terminal in Visual Studio Code, create a new Angular project with:

```
ng new directivesdemo
```

Press enter to accept the default values for all the question prompts that follow

Navigate into the root project folder from the terminal with:

```
cd directivesdemo
```

Build the app and serve it via Angular's live development server by typing:

```
ng serve
```

The final output line from this should indicate that the compilation process was successful and identify the port that the live development server is currently serving up the Angular app dynamically from (by default this will be port 4200)

Open a browser tab at:
http://localhost:4200/

You should be able to see the default landing page for all new autogenerated Angular projects.

To stop the development server, type Ctrl+C in the terminal window that it is running in. You can restart again anytime by typing `ng serve` in the project root folder.

3 ngClass

The source code for this lab can be found in the `changes` subfolder of `Directives-Demo`

NgClass is conceptually similar to standard `[class]` binding: it permits dynamic binding of CSS classes to a specified HTML element

Modify the following files in `src/app` with the latest update from `changes`:

`app.component-v1.ts`

`app.component-v1.html`

`app.component-v1.css`

The template expression that is used with `[ngClass]` can return all these items:

- A space-delimited string of class names.
- An object with class names as the keys and truthy or falsy expressions as the values.
- An array of class names

and is thus functionally identical to the multiple class binding `[class]` syntax that we used for property binding in an earlier lab.

Verify that the classes on the 3 paragraphs change accordingly. Notice that for the 3rd case where there are two classes with conflicting style rules now applied to the <p> element simultaneously (`normal` and `medium`), the style rule for the class that appears last in the list of class names takes effect.

You can change the truthy/falsy values of the object used in the 2nd approach and verify that the correct classes appear on the affected element.

3.1 Difference between [class] and [ngClass]

To add or remove a single class, it is recommended to use the `[class]` binding covered in a previous lab.

Modify the following files in `src/app` with the latest update from `changes`:

`app.component-v1-2.ts`

`app.component-v1-2.html`

Test out making dynamic changes to the classes applied to the various <p> elements by clicking on the corresponding buttons.
For the text box with the class name to add to the 3rd paragraph, try adding / removing the class names listed in `app.component.css` to see the effect (for e.g. `large, special, emphasize, danger, safe`).

Here, we can see the primary difference between using `[ngClass]` and standard `[class]` binding to dynamically set class names on a element. For the case when we are using an object with class names as the keys and truthy or falsy expressions as the values, using `[ngClass]` allows dynamic updating of the classes when any of the object key values changes while standard `[class]` binding does not.

This is because the standard `[class]` binding requires a change in the identity of the object used in the template expression in order for there to be a dynamic change in the binding during run time. If you only change a property within the object, there will be no update in the binding.

References:

https://v17.angular.io/guide/built-in-directives#ngClass
https://www.tektutorialshub.com/angular/angular-ngclass-directive/
https://www.netjstech.com/2020/04/angular-ngclass-directive-with-examples.html

4 ngStyle

NgStyle is conceptually similar to standard `[style]` binding: it permits dynamic binding of CSS properties to a specified HTML element

Modify the following files in `src/app` with the latest update from `changes`:

`app.component-v2.ts`

`app.component-v2.html`

For `[ngStyle]` binding, we can only use an object whose key and values are style name and style values respectively. You cannot assign a string containing these style rules as was the case in normal `[style]` binding that we studied in a previous lab.

Verify that changing the value of the style properties in the object causes the inline styles to be updated dynamically.

References:

https://codecraft.tv/courses/angular/built-in-directives/ngstyle-and-ngclass/#_ngstyle
https://www.netjstech.com/2020/04/angular-ngstyle-directive-with-examples.html
https://www.tektutorialshub.com/angular/angular-ngstyle-directive/

5 Using @if and @else (Angular 17 and later)

In Angular 17 and later, @if and @else are block directives which used to implement control flow syntax (known as "declarative control flow" or "template control flow blocks"), which provides a more readable and modern alternative to the traditional *ngIf, *ngFor used in earlier versions of Angular.

@if is a new block directive that allows you to conditionally render specific HTML portions of your template based on an expression. @else is used in conjunction with @if to display an alternate HTML snippet when the condition is false. They both implement the same conceptual functional of the if-else statements found in conventional programming languages

Modify the following files in `src/app` with the latest update from `changes`:

`app.component-v3.ts`

`app.component-v3.html`

Experiment with the first checkbox, second checkbox and numeric age field to see the 3 forms of this syntax in action.

References
https://www.geeksforgeeks.org/angular-17-structural-directives/#1-ngif

6 Using @for (Angular 17 and later)

The @for block directive iterates over a data collection (e.g. array, list, etc) and creates snippet of HTML (containing one or more elements) for each item in the collection. It helps to simplify the construction of HTML blocks which involve repetition of standard elements (e.g. lists or tables). This is available in Angular 17 and later and is part of Angular's new declarative control flow system designed to improve template readability as compared the traditional *ngFor in Angular 17 and earlier.

Modify the following files in `src/app` with the latest update from `changes`:

`app.component-v4.ts`

`app.component-v4.html`

`app.component-v4.css`

Add this file to `src/app` from `changes`:

`Employee.ts`

This represents a sample domain model class, which models application data for the business domain that the app is being built for.

In th expression:
`@for (animal of animals; track animal)`
`animal` is called the template input variable (this is different from a template reference variable). This represents the currently iterated item from the collection `animals`. The scope of the template input variable item is within the HTML block nested inside the @for. You can access it anywhere within that, but not outside of it.

References:
https://blog.angular-university.io/angular-for/

6.1 Accessing index of item in collection

Modify the following files in `src/app` with the latest update from `changes`:

`app.component-v5.ts`

`app.component-v5.html`

`app.component-v5.css`

@for exposes several loop contextual variables, which help us to fine-tune the display. These variables are all preceded with a $. We can assign these values to a local variable and use it in the nested HTML block.

- `$index: number`: The zero-based index of the current element in the collection.
- `$count: number`: The total no of items in the collection
- `$first: boolean`: is set to True when the item is the first item in the collection.
- `$last: boolean`: is set to True, when the item is the last item in the collection.
- `$even: boolean`: is set to True when the item has an even index in the collection.
- `$odd: boolean`: is set to True when the item has an odd index in the collection.

Here we set the local variable i to the `$index` and use this in the HTML block to allow us to keep track of which button has been clicked on.

6.2 Styling using first, last, odd and even loop contextual variables

Modify the following files in `src/app` with the latest update from `changes`:

`app.component-v6.html`

`app.component-v6.css`

Here we use the `$first`, `$last`, `$even` and `$odd` exported values to style the `<div>` surrounding the respective buttons

6.3 @for with child components

Open a new command prompt in the root folder of the project created earlier, and type this command to generate a new component:

```
ng generate component Employee
```

Modify the following files in `src/app` with the latest update from `changes`:

`app.component-v7.html`

`app.component-v7.ts`

`employee.component-v7.ts`

`employee.component-v7.html`

`employee.component-v7.css`

In the root template, we use a @for to iterate through the Employee objects in the array, passing each object as well as its index in the collection to a newly created child component `<app-employee>` via two separate property bindings.

6.4 Passing values from child components back to parent component

Make the following changes:

`app.component-v8.html`

`app.component-v8.ts`

`app.component-v8.css`

`employee.component-v8.ts`

`employee.component-v8.html`

`employee.component-v8.css`

Next, we provide appropriate input fields in the child template to capture new values for the Employee object passed down to the respective child component.

Notice that we use NgModel for two-way binding for both the `name` and `age` properties of the Employee object, which results in a change in the original object in the array in the parent component since data is passed by reference.

However, we are not able to perform two-way binding in this way for radio buttons, so we need a event handler method in the child component that checks whether the married radio button is ticked to set the new value for the `isMarried` property.

The index of the employee object from the original array that was changed in the child template is transmitted back in the event binding from the child component so that this can be used in the dynamic binding in the `[ngClass]` directive to highlight the specific row that was changed.

7 Combining @if and @for

Modify the following files in `src/app` with the latest update from `changes`:

`app.component-v9.html`

`app.component-v9.css`

`app.component-v9.ts`

Here we have a @for nested within an @if, so either the entire table contents is either shown in its entirety or not shown at all.

7.1 Switching sequence of @if and @for

Modify the following files in `src/app` with the latest update from `changes`:

`app.component-v10.html`

`app.component-v10.ts`

Here we have a @if nested within a @for, so only specific employee objects are rendered in the table rows, depending on the evaluation of a conditional expression.

8 Using @switch, @case and @default (Angular 17 and later)

Modify the following files in `src/app` with the latest update from `changes`:

`app.component-v11.html`

`app.component-v11.ts`

@switch, @case and @default directives are used together in conjunction to provide the functionality of a standard switch statement in conventional programming languages.

Some points to bear in mind:

- The @case and @default directives must be nested within the @switch directive

- Angular will render all HTML blocks for all @case whose evaluated expression values match the evaluated @switch expression values
- The HTML blocks of non-matching @case are not hidden, but removed from the DOM
- If there are no matches for any @case, the HTML block associated with the @default directive is displayed
- You can place one @default directive anywhere inside the container element and not necessarily at the bottom.

9 Migrating from older directives to new @if, @for and @switch syntax

Modify the following files in `src/app` with the latest update from `changes`:

`app.component-v12.html`

`app.component-v12.css`

`app.component-v12.ts`

This demonstrates the use of the older *ngIf and *ngFor directives in earlier versions of Angular prior to Angular 17. These directives have now been replaced by the @if and @for block directives. Verify that these older directives still work.
To migrate this code base to use the new @if, @for, @switch syntax, first stop the development server and then close the VS code instance where this project folder is open in.

Then type into a command prompt in the root folder of the project:

```
ng generate @angular/core:control-flow
```

Select the following options:

```
? Which path in your project should be migrated? ./
? Should the migration reformat your templates? Yes
   IMPORTANT! This migration is in developer preview. Use with caution.
```

Open the project folder again in VS Code, and verify that the code has indeed been refactored in `app.component.hml`. Start the development server and verify as well that the functionality is still identical as prior to the refactoring.